УГНТУ • ЦЕНТР ТОП ИТ • 09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА

Программирование и прототипирование ИТ-решений

Функциональное программирование функции, аргументы, LEGB, lambda

ЛЕКЦИЯ 9

Ахметов Ильнур Вазирович

План лекции

1. Определение функций (def), передача параметров и оператор return.
2. Позиционные и именованные аргументы, *args и **kwargs.
3. Области видимости переменных: правило LEGB.
4. Анонимные функции — lambda-выражения.
5. Функциональный стиль: функции как значения, map/filter/reduce, композиция.
6. Ловушки: изменяемый default, затенение имён, позднее связывание.

В конце — вопросы для закрепления (по одному на слайд).

ПОВТОРЕНИЕ

Что мы уже умеем

- Лекция 7: списки — поиск, сортировка, map/filter.
- Лекция 8: хеш-таблицы — dict и set, O(1).
- Мы всё время пользовались ЧУЖИМИ функциями:
- len(), sum(), sorted(), map(), print().
- Сегодня научимся писать свои и поймём,
- как они устроены внутри: аргументы, области видимости.
- И перейдём к функциональному стилю:
- функция как значение, которое можно передать.

консоль Python

```
>>> def oil_rate(q_liq, wct=0.0):
...     return q_liq * (1 - wct)
...
>>> oil_rate(120.0, 0.35)
78.0
>>> oil_rate(80.0)
80.0

# своя функция — как встроенная
```

Зачем нужны функции

- DRY: одинаковый расчёт не копируется по коду.
- Декомпозиция: большая задача = маленькие шаги.
- Имя = документация: oil_rate() читается без комментария.
- Тестируемость: функцию можно проверить отдельно.
- Локальность изменений: поправил в одном месте.
- Повторное использование: один раз написал — везде вызвал.
- Функция — единица мышления, а не «синтаксический сахар».

плохо / хорошо

```
# плохо: копипаста в 20 местах
oil_1 = q1 * (1 - w1)
oil_2 = q2 * (1 - w2)
oil_3 = q3 * (1 - w3)

# хорошо: одна функция
def oil_rate(q_liq, wct):
    return q_liq * (1 - wct)

oil = [oil_rate(q, w) for q, w in rows]
```

Определение функции: def

- def имя(параметры): — заголовок, далее тело с отступом.
- def — оператор, выполняется как и любой другой:
- он СОЗДАЁТ объект-функцию и связывает её с именем.
- Параметры — имена в заголовке; аргументы — значения при вызове.
- Тело выполняется только при ВЫЗОВЕ, не при определении.
- Имя функции — глагол или существительное-действие:
- oil_rate, calc_rate, validate_row.
- Определение можно вызвать до его текста? Нет — только после
- (в модуле порядок важен; внутри функции — при вызове).

определение и вызов

```
def oil_rate(q_liq, wct):
    """Дебит нефти: жидкость * (1 -
обводнённость)."""
    oil = q_liq * (1 - wct)
    return round(oil, 2)

# вызов:
print(oil_rate(120.0, 0.35))
78.0

# тело НЕ выполнялось при def:
print(oil_rate)  # <function oil_rate at ...>
```

return: что возвращает функция

- return завершает функцию и отдаёт значение вызывающему.
- Нет return (или return без значения) — функция вернёт None.
- return a, b — это ОДНО значение: кортеж (a, b).
- После return код в функции не выполняется.
- print() — побочный эффект в консоль, НЕ возврат значения;
- print возвращает None — классическая путаница.
- Паттерн «успех/данные»: return True, value
- или return None при неудаче.

консоль Python

```
>>> def f(x):
...     return x * 2
>>> f(5)
10

>>> def g(x):
...     print(x)          # печать, не return
>>> g(5)
5
>>> g(5) is None
True

>>> def h(x):
...     return x, x * 2   # кортеж!
>>> h(5)
(5, 10)
```

Ранний выход: guard clauses

- Проверки — в начале функции, каждая завершает её при ошибке.
- Так тело остаётся плоским: без вложенных if на 3 уровня.
- Условие «плохой вход» → return None / raise.
- Читается сверху вниз: сначала «что не так»,
- потом «основная логика».
- raise ValueError — когда ошибка должна быть замечена,
- return None — когда отсутствие данных нормально.

```
guard clauses
def oil_rate(q_liq, wct):
    if q_liq is None:
        return 0.0
    if not (0.0 <= wct <= 1.0):
        raise ValueError('wct вне 0..1')
    if q_liq <= 0:
        return 0.0
    # основная логика — на верхнем уровне
    return round(q_liq * (1 - wct), 2)
```

Docstring и аннотации типов

- Docstring — строка сразу после def, в тройных кавычках.
- help(f) и f.__doc__ показывают её; это документация,
- доступная из кода.
- Аннотации: def f(q: float, w: float = 0.0) -> float.
- Аннотации НЕ проверяются в рантайме — это подсказка
- для человека, IDE и mypy.
- Хорошая функция: имя + docstring + аннотации = контракт.
- Одна-две строки docstring достаточно для учебного кода.

```
полное определение
def oil_rate(q_liq: float,
             wct: float = 0.0) -> float:
    """Вернуть дебит нефти.

    q_liq — дебит жидкости, т/сут
    wct   — обводнённость, 0..1
    """
    return round(q_liq * (1 - wct), 2)

>>> oil_rate.__annotations__
{'q_liq': float, 'wct': float, 'return': float}
```

Функция — объект первого класса

- В Python функция — такой же объект, как число или строка.
- Её можно присвоить переменной, положить в список или словарь,
- передать аргументом и вернуть из другой функции.
- f.__name__ — имя; type(f) — function.
- Словарь-диспетчер: {'oil': oil_rate, 'liq': liquid_rate} —
- замена длинному if/elif по строке.
- Именно из этого свойства вырастает весь функциональный стиль.

```
функция как значение
def oil_rate(q, w): return q * (1 - w)

calc = oil_rate        # присвоили
calc(120.0, 0.35)
78.0

ops = {'oil': oil_rate, 'liq': lambda q, w: q}
ops['oil'](120.0, 0.35)
78.0

oil_rate.__name__      # 'oil_rate'
```

Позиционные аргументы

- При вызове f(a, b) значения подставляются ПО ПОРЯДКУ
- в параметры заголовка: первый → первый, второй → второй.
- Это самый частый и самый компактный способ.
- Порядок критичен: перепутали q_liq и wct — получили чушь,
- и Python не заметит (оба float).
- Правило защиты: если у функции 2+ параметра одного типа —
- подумайте об именованной передаче при вызове.

```
порядок имеет значение
def oil_rate(q_liq, wct):
    return q_liq * (1 - wct)

oil_rate(120.0, 0.35)   # верно
78.0

oil_rate(0.35, 120.0)   # перепутали!
-41.65

# ошибки нет — просто неверный ответ
```

Именованные аргументы

- f(wct=0.35, q_liq=120.0) — порядок не важен, имена — да.
- Читаемость: в вызове видно, что чему передаётся.
- Правило: позиционные аргументы идут ДО именованных.
- f(120.0, wct=0.35) — можно; f(q_liq=120.0, 0.35) — нельзя,
- это SyntaxError.
- Именованные аргументы — основной способ вызова
- для функций с 3+ параметрами и для «флагов».

консоль Python

```
def oil_rate(q_liq, wct):
    return round(q_liq * (1 - wct), 2)

>>> oil_rate(120.0, 0.35)
78.0
>>> oil_rate(wct=0.35, q_liq=120.0)
78.0
>>> oil_rate(120.0, wct=0.35)
78.0
>>> oil_rate(q_liq=120.0, 0.35)
SyntaxError: positional argument follows keyword
```

Значения по умолчанию

- def f(q, wct=0.0) — если аргумент не передан, берётся 0.0.
- Параметры с default идут ПОСЛЕ параметров без него.
- Default вычисляется ОДИН РАЗ — при определении функции,
- а не при каждом вызове. Это ключ к следующей ловушке.
- Хороший default: неизменяемое (0, 0.0, '', None, ()).
- Default делает функцию удобной: типичный случай коротко,
- редкий случай — явно.

консоль Python

```
def oil_rate(q_liq, wct=0.0):
    return round(q_liq * (1 - wct), 2)

>>> oil_rate(80.0)          # default
80.0
>>> oil_rate(80.0, 0.5)     # явно
40.0
>>> oil_rate(wct=0.5, q_liq=80.0)
40.0

# default вычислен при def — один раз:
>>> oil_rate.__defaults__
(0.0,)
```

Ловушка: изменяемый default

- def push(x, acc=[]) — список создаётся ОДИН РАЗ,
- при определении, и живёт между вызовами.
- Второй вызов уже видит результат первого — накопление.
- Это не баг Python, а следствие «default вычисляется
- один раз».
- Лечение: default=None, а внутри — if acc is None:
- acc = []. Каждый вызов получает свежий список.
- Правило: никогда не ставьте list/dict/set в default.

консоль Python

```
>>> def push(x, acc=[]):
...     acc.append(x); return acc
>>> push(1)
[1]
>>> push(2)          # тот же список!
[1, 2]

# правильно:
>>> def push2(x, acc=None):
...     if acc is None: acc = []
...     acc.append(x); return acc
>>> push2(1), push2(2)
([1], [2])
```

*args: переменное число позиционных

- def f(*args): — все позиционные собираются в КОРТЕЖ args.
- Имя args — соглашение; важно звёздочка.
- Внутри: len(args), args[0], sum(args), перебор.
- Вызов: f(), f(1), f(1, 2, 3) — все валидны.
- Типовые применения: обёртки над print, агрегаторы,
- функции «для любого числа значений».
- Если *args есть, параметры с default идут ДО него.

консоль Python

```
>>> def sum_all(*args):
...     print(type(args), args)
...     return sum(args)
>>> sum_all(1, 2, 3, 4)
<class 'tuple'> (1, 2, 3, 4)
10
>>> sum_all()
<class 'tuple'> ()
0

>>> def mean(*xs):
...     return sum(xs) / len(xs) if xs else 0.0
>>> mean(85.4, 12.7, 65.3)
54.46666666666667
```

**kwargs: переменное число именованных

- def f(**kwargs): — именованные собираются в СЛОВАРЬ.
- Ключи словаря — имена аргументов, значения — значения.
- Вызов: f(a=1, b=2); f() — пустой словарь.
- Типовые применения: функции с большим числом опций,
- обёртки-логгеры, передача настроек «через» функцию.
- Порядок в сигнатуре: positional, default, *args, keyword-only, **kwargs.
- Имена args/kwargs — соглашение, но придерживайтесь его.

```
консоль Python
>>> def describe(**kw):
...     for k, v in kw.items():
...         print(k, '=', v)
>>> describe(q_liq=120.0, wct=0.35)
q_liq = 120.0
wct = 0.35

>>> def log(msg, **opts):
...     return f"{msg} {opts}"
>>> log('старт', level='INFO', n=3)
"старт {'level': 'INFO', 'n': 3}"
```

Распаковка при вызове

- f(*[1, 2]) — звёздочка в ВЫЗОВЕ распаковывает список
- в позиционные аргументы.
- f(**{'a': 1}) — двойная звёздочка распаковывает словарь
- в именованные аргументы.
- Симметрия: * при определении — собрать, * при вызове — раздать.
- Типовые применения: f(*row) для строки из CSV,
- f(**config) для словаря настроек.
- Распаковка работает с любым итерируемым, ** — только со словарём.

консоль Python

```
>>> def oil_rate(q_liq, wct):
...     return round(q_liq * (1 - wct), 2)

>>> row = (120.0, 0.35)
>>> oil_rate(*row)
78.0

>>> cfg = {'q_liq': 120.0, 'wct': 0.35}
>>> oil_rate(**cfg)
78.0

>>> print(*[1, 2, 3], sep=' | ')
1 | 2 | 3
```

Порядок и keyword-only параметры

- Полный порядок: def f(a, b=1, *args, c, d=2, **kw).
- Одиночная * в сигнатуре делает всё после неё
- KEYWORD-ONLY: передать можно только по имени.
- Зачем: защита от случайной перестановки аргументов.
- Слэш / (Python 3.8+) делает параметры POSITIONAL-ONLY:
- def f(a, /, b) — a только позиционно (так у len, print).
- Встроенные функции стесняются: len(x, /) — не len(obj=x).

```
keyword-only и positional-only
def f(a, b=1, *args, c, d=2, **kw):
    return a, b, args, c, d, kw

>>> f(1, 2, 3, 4, c=5, e=6)
(1, 2, (3, 4), 5, 2, {'e': 6})

>>> f(1, 2, 3, 4, 5)   # c не передан
TypeError: missing keyword-only argument: 'c'

>>> def g(a, /, b): return a + b
>>> g(1, b=2)      # 3
>>> g(a=1, b=2)    # TypeError
```

Как передаются аргументы

- В Python аргументы передаются «по ссылке на объект»
- (call by object reference / call by sharing).
- Внутри функции имя ссылается на ТОТ ЖЕ объект.
- Изменяемый объект можно изменить внутри функции —
- изменения видны снаружи (list.append).
- Переприсваивание имени внутри функции НЕ меняет
- внешнюю переменную — меняется лишь локальная ссылка.
- Для неизменяемых (int, str, tuple) «изменение» всегда
- означает переприсваивание — снаружи ничего не произойдёт.

консоль Python

```
>>> def add(xs):
...     xs.append(99)       # меняем объект
>>> data = [1, 2]
>>> add(data); data
[1, 2, 99]

>>> def rebind(xs):
...     xs = [0]            # новая ссылка
>>> data2 = [1, 2]
>>> rebind(data2); data2
[1, 2]
```

Правила работы с изменяемыми аргументами

- Правило 1: функция не должна молча менять входные данные.
- Правило 2: нужен результат — верните новый объект
- (sorted, а не sort; новый список, а не append в чужой).
- Правило 3: если изменение — смысл функции, назовите её так:
- sort_in_place(), fill_buffer().
- Правило 4: копия на входе — list(xs), dict(d), xs[:] —
- если вход пришёл снаружи и его нельзя портить.
- Python-идиома: sorted() возвращает, .sort() мутирует.

```
мутировать или вернуть
rows = [('K-13', 12.7), ('K-12', 85.4)]

# мутирует вход, возвращает None:
rows.sort(key=lambda r: r[1])

# возвращает новый список:
ordered = sorted(rows, key=lambda r: r[1])

# копия, если вход чужой:
def clean(xs):
    out = list(xs)
    out.sort()
    return out
```

Шпаргалка: сигнатура целиком

1. def f(a, b): — позиционные; вызывать по порядку.
2. def f(a, b=1): — default; параметры с default идут последними.
3. def f(*args): — кортеж всех позиционных.
4. def f(*, c): — keyword-only: только по имени.
5. def f(**kw): — словарь всех именованных.
6. def f(a, /, b): — positional-only (Python 3.8+).
7. Вызов: f(*seq), f(**d) — распаковка; f(1, x=2) — смешанный.
8. Правило чтения: слева — что обязательно, справа — что опционально.

Правило LEGB

- L — Local: имена внутри текущей функции.
- E — Enclosing: имена внешней (объемлющей) функции.
- G — Global: имена уровня модуля.
- B — Built-in: встроенные имена Python (len, print, list).
- Поиск идёт строго в этом порядке, первое найденное — побеждает.
- Правило действует при ЧТЕНИИ имени.
- Присваивание всегда создаёт ЛОКАЛЬНОЕ имя (кроме global/nonlocal).

```
порядок поиска
x = 'global'            # G

def outer():
    x = 'enclosing'     # E
    def inner():
        x = 'local'     # L
        print(x)        # L -> 'local'
    inner()
outer()

# убери строку x = 'local' -> 'enclosing'
# убери и в outer -> 'global'
```

L: локальная область

- Локальные имена создаются при вызове функции и исчезают
- при выходе из неё — у каждого вызова своя область.
- Параметры — тоже локальные имена.
- Рекурсивный вызов получает СВОЮ локальную область.
- locals() показывает словарь локальных имён —
- удобно для отладки, в коде не использовать.
- Изоляция — благо: одинаковые имена в разных функциях
- не конфликтуют.

```
консоль Python
>>> def f(q):
...     tmp = q * 2      # локальные: q, tmp
...     return tmp
>>> f(5)
10
>>> tmp              # снаружи нет
NameError: name 'tmp' is not defined

>>> def g():
...     x = 1
...     return locals()
>>> g()
{'x': 1}
```

E: enclosing и замыкания

- Вложенная функция ВИДИТ имена внешней функции —
- это область Enclosing.
- Если outer вернула inner, inner продолжает помнить
- переменные outer — это ЗАМЫКАНИЕ (closure).
- Замыкание = функция + «захваченные» значения.
- Так получают функции с настройкой без классов:
- make_scaler(0.85) → функция, умножающая на 0.85.
- Каждое замыкание имеет СВОЁ состояние, независимое от других.

замыкание

```
def make_scaler(k):
    def scale(x):
        return round(x * k, 2)   # k из E
    return scale

>>> k85 = make_scaler(0.85)
>>> k85(100.0)
85.0
>>> k50 = make_scaler(0.5)
>>> k50(100.0), k85(100.0)
(50.0, 85.0)
>>> k85.__closure__[0].cell_contents
0.85
```

G: глобальная область и global

- Имена уровня модуля видны во всех функциях — на чтение.
- Присваивание внутри функции создаёт ЛОКАЛЬНОЕ имя,
- даже если глобальное с таким именем существует.
- global x в функции разрешает ЗАПИСЬ в глобальное имя.
- global нужен редко: он делает поведение функции
- зависимым от внешнего состояния и трудно тестируемым.
- Лучше: вернуть новое значение и присвоить его снаружи.

```
global: как и почему не надо
counter = 0

def bump():
    global counter
    counter += 1

>>> bump(); bump(); counter
2

# лучше: без global
def bumped(n):
    return n + 1
counter = bumped(counter)
```

B: built-in и затенение

- Последняя инстанция поиска — встроенные имена.
- Если имя не найдено нигде, Python ищет среди built-in:
- len, sum, list, dict, print, max, id, type.
- Затенение (shadowing): своя переменная с именем list
- перекрывает встроенную функцию — и list() перестаёт работать.
- Не называйте переменные: list, dict, set, sum, min, max,
- type, id, input, str, int, next, file.
- Ловушка особенно частая в учебном коде: sum = 0 в цикле.

консоль Python

```
>>> list = [1, 2, 3]
>>> list()
TypeError: 'list' object is not callable

>>> sum = 0
>>> for x in [1, 2]:
...     sum = sum + x
>>> sum([1, 2])       # уже не функция
TypeError: 'int' object is not callable

>>> import builtins
>>> builtins.list is list   # True до затенения
```

nonlocal и UnboundLocalError

- nonlocal x — запись в имя внешней (Enclosing) функции.
- Без nonlocal x += 1 внутри inner создаст локальное x
- и упадёт с UnboundLocalError на чтении.
- UnboundLocalError: имя локальное (есть присваивание в теле),
- но прочитано до присваивания.
- Типичный сценарий: счётчик внутри замыкания.
- global — для модуля, nonlocal — для внешней функции,
- ничего — для локальной.

```
nonlocal
def make_counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc

>>> c = make_counter()
>>> c(), c(), c()
(1, 2, 3)

# без nonlocal:
UnboundLocalError: cannot access local
variable 'n' where it is not associated
```

Замыкания на практике

- Замыкание хранит настройку и «умеет» её применять:
- порог дебита, коэффициент, формат вывода.
- Плюсы: нет класса ради одного метода; состояние
- инкапсулировано; удобно передавать как аргумент.
- Минусы: состояние не видно (.__closure__), сложнее отладка,
- и легко попасть в позднее связывание (следующий блок).
- Где встречается: декораторы, фабрики функций, кэши,
- валидаторы с порогом.

```
порог как замыкание
def above(threshold):
    def check(q):
        return q > threshold
    return check

high = above(50.0)
low = above(10.0)

>>> high(85.4), low(85.4)
(True, True)
>>> high(12.7), low(12.7)
(False, True)

# порог «зашит» в функцию — без глобальных
```

Анонимная функция: синтаксис

- lambda параметры: выражение — функция без имени.
- Тело — РОВНО ОДНО выражение; return не пишется,
- результат выражения и есть возвращаемое значение.
- Нельзя: присваивание, несколько операторов, блок if,
- цикл. Можно: условное выражение a if c else b.
- Объект-функция создаётся в момент выполнения lambda.
- lambda q, w: q * (1 - w) — та же функция, что def,
- только без имени.

консоль Python

```
>>> f = lambda q, w: q * (1 - w)
>>> f(120.0, 0.35)
78.0
>>> f.__name__
'<lambda>'

# то же через def:
>>> def g(q, w):
...     return q * (1 - w)

# условное выражение внутри lambda:
>>> ok = lambda q: 'норма' if q > 50 else 'мало'
>>> ok(85.4), ok(12.7)
('норма', 'мало')
```

key= в sorted, min, max

- Главное место lambda в аналитике — параметр key=.
- sorted(rows, key=lambda r: r[1]) — сортировка по полю.
- min/max(…, key=…) — «самый тяжёлый/лёгкий» по правилу.
- Не путать key= с reverse=: key — ЧТО сравнивать,
- reverse — в каком порядке.
- Вложенные записи: key=lambda r: (r['куст'], -r['дебит']) —
- кортеж задаёт порядок по двум полям сразу.
- Это продолжение сортировки из лекции 7 — там был тот же key=.

```
key= на практике
rows = [('К-12', 85.4), ('К-13', 12.7)]

>>> sorted(rows, key=lambda r: r[1])
[('К-13', 12.7), ('К-12', 85.4)]
>>> sorted(rows, key=lambda r: r[1],
reverse=True)
[('К-12', 85.4), ('К-13', 12.7)]

>>> max(rows, key=lambda r: r[1])
('К-12', 85.4)

# два поля: куст по возрастанию, дебит по
убыванию
>>> key=lambda w: (w['куст'], -w['дебит'])
```

map и filter с lambda

- map(f, seq) применяет f к каждому элементу —
- возвращает ИТЕРАТОР, а не список.
- filter(f, seq) оставляет элементы, где f вернула True.
- Оба ленивы: работа происходит при обходе.
- Нужен список — оберните: list(map(...)), list(filter(...)).
- В современном Python часто читаемее генераторные
- выражения — сравнение на слайде ФП · 7/9.
- lambda + map/filter — это «функциональное ядро» в одну строку.

консоль Python

```
>>> qs = [120.0, 0.0, 80.0, 200.0]

>>> oils = list(map(lambda q: round(q * 0.65, 2),
qs))
[78.0, 0.0, 52.0, 130.0]

>>> active = list(filter(lambda q: q > 0, qs))
[120.0, 80.0, 200.0]

>>> m = map(lambda q: q * 0.65, qs)
>>> next(m)
78.0
>>> type(m)
<class 'map'>
```

Несколько аргументов и lambda в структурах

- lambda может принимать сколько угодно параметров:
- lambda q, w: q * (1 - w).
- functools.reduce(lambda a, b: a + b, xs) — свёртка списка
- в одно значение.
- lambda в словаре-диспетчере: {'oil': lambda q, w: ...} —
- таблица операций вместо ветвлений.
- lambda как значение по умолчанию — почти всегда ошибка:
- трудно читать и отлаживать.
- Правило: lambda уместна там, где функция нужна ОДИН раз
- и её тело — одно короткое выражение.

```
выражения с двумя аргументами
from functools import reduce

>>> reduce(lambda a, b: a + b, [1, 2, 3, 4])
10

>>> ops = {
...     'oil': lambda q, w: q * (1 - w),
...     'liq': lambda q, w: q,
... }
>>> ops['oil'](120.0, 0.35)
78.0

>>> pairs = [(1, 2), (3, 4)]
>>> list(map(lambda p: p[0] * p[1], pairs))
[2, 12]
```

Ловушка позднего связывания

- lambda захватывает ИМЯ, а не значение.
- [lambda: i for i in range(3)] — все три функции смотрят
- на одну переменную i, а к моменту вызова она равна 2.
- Итог: [2, 2, 2], а не [0, 1, 2].
- Лечение: зафиксировать значение через default-параметр:
- lambda i=i: i.
- Или не использовать lambda в цикле — собрать через def
- или генератор.
- Это то же правило «default вычисляется при определении».

консоль Python

```
>>> funcs = [lambda: i for i in range(3)]
>>> [f() for f in funcs]
[2, 2, 2]           # не [0, 1, 2]!

# правильно: значение в default
>>> funcs = [lambda i=i: i for i in range(3)]
>>> [f() for f in funcs]
[0, 1, 2]

# или без lambda:
>>> funcs = [(lambda n: (lambda: n))(i) for i in
range(3)]
```

Когда lambda вредит

- PEP 8 прямо не рекомендует присваивать lambda имени:
- f = lambda x: x + 1 — вместо этого def f(x).
- Причина: имя <lambda> в трейсбеке — отладка вслепую.
- Сложное выражение внутри lambda нечитаемо:
- если нужна ветка if/else на 2 уровня — пишите def.
- lambda нельзя документировать: нет docstring.
- lambda внутри lambda работает, но понимать это будет
- только автор — и то через месяц нет.
- Правило: lambda — для key= и одноразовых коротких колбэков.

антипаттерны

```
# плохо: lambda с именем
rate = lambda q, w: q * (1 - w)

# хорошо:
def rate(q, w):
    return q * (1 - w)

# плохо: нечитаемая вложенность
key=lambda r: r[1] if r[1] > 0 else (r[2] or -1)

# хорошо: именованная функция
def sort_key(r):
    return r[1] if r[1] > 0 else (r[2] or -1)
```

lambda или def: как выбрать

lambda одно выражение, нужно один раз, передаётся как аргумент (key=, колбэк).

def нужно имя, docstring, несколько операторов, ветвления, отладка.

правило если функцию хочется назвать — ей нужен def.

сравнение

```
sorted(rows, key=lambda r: r[1])          # уместно: одноразовый ключ
def by_rate(r): return r[1]                # уместно: ключ нужен в 3+ местах
sorted(rows, key=by_rate)                  # читается и отлаживается
```

Функции как значения

- Функциональное программирование начинается с простого факта:
- функция — такое же значение, как число.
- Её можно передать в функцию и вернуть из функции.
- Функция, принимающая или возвращающая функцию, называется
- ФУНКЦИЕЙ ВЫСШЕГО ПОРЯДКА.
- Так работают map, filter, sorted — их «поведение»
- задаётся аргументом-функцией.
- Это даёт расширяемость без копирования кода:
- один каркас — разные правила.

передача функции

```
def apply(f, xs):
    return [f(x) for x in xs]

def half(x):  return round(x / 2, 2)
def sq(x):    return round(x * x, 2)

>>> apply(half, [10, 20])
[5.0, 10.0]
>>> apply(sq, [10, 20])
[100.0, 400.0]

# каркас один, поведение — параметр
```

Чистые функции и побочные эффекты

- Чистая функция: результат зависит ТОЛЬКО от аргументов,
- и она ничего не меняет снаружи.
- Плюсы: предсказуема, тестируема, безопасна при параллельном
- вызове, результат можно кэшировать.
- Побочный эффект: print, запись в файл, изменение списка,
- global-переменная.
- Практика аналитика: расчёт — чистая функция;
- чтение/запись — отдельный слой.
- Идеал не в «всё чистое», а в «эффекты собраны в одном месте».

чистая vs с эффектом

```
# чистая:
def oil_rate(q, w):
    return round(q * (1 - w), 2)

# не чистая: меняет вход и печатает
def bad(rows):
    for r in rows:
        r[1] *= 0.65
        print(r)

# кэш возможен только для чистой:
from functools import lru_cache
@lru_cache(maxsize=None)
def oil_rate_c(q, w): return round(q * (1 - w),
2)
```

map и filter: функциональное ядро

- map(f, xs) — «применить ко всем».
- filter(f, xs) — «оставить подходящие».
- Обе возвращают итераторы (ленивые) и не меняют вход.
- Комбинация: list(map(f, filter(p, xs))) — конвейер.
- Встроенные способы без lambda: map(str, xs),
- filter(None, xs) — отбрасывает falsy-значения.
- Читаемость: одна простая функция — map/filter;
- сложное условие — генераторное выражение.

```
конвейер map/filter
qs = [120.0, 0.0, 80.0, 200.0, -5.0]

>>> list(filter(lambda q: q > 0, qs))
[120.0, 80.0, 200.0]

>>> oils = list(map(lambda q: round(q * 0.65, 2),
...                 filter(lambda q: q > 0, qs)))
[78.0, 52.0, 130.0]

>>> list(filter(None, [0, 1, '', 'a', None]))
[1, 'a']
```

reduce: свёртка в одно значение

- reduce(f, xs, init) последовательно «сворачивает» список:
- f(f(f(init, x1), x2), x3).
- Первые два аргумента f — накопленное значение и очередной
- элемент.
- Для сумм/произведений есть sum/max/min — они читаемее
- и быстрее: reduce(sum) — редкость.
- reduce незаменим для свёртки структур:
- слияние словарей, склейка строк, поиск «лучшего» по правилу.
- В Python 3 reduce живёт в functools, а не в built-in.

```
консоль Python
from functools import reduce

>>> reduce(lambda a, b: a + b, [1, 2, 3, 4])
10
>>> reduce(lambda a, b: a * b, [1, 2, 3, 4])
24

# с начальным значением:
>>> reduce(lambda a, b: a + b, [1, 2], 100)
103

# слияние словарей свёрткой:
>>> reduce(lambda d, e: {**d, **e}, [{'a': 1},
{'b': 2}])
{'a': 1, 'b': 2}
```

operator: key без lambda

- Для «взять поле» lambda — самый частый, но не лучший случай.
- operator.itemgetter(1) — функция, берущая элемент по индексу;
- itemgetter('дебит') — по ключу словаря.
- operator.attrgetter('q') — по атрибуту объекта.
- Плюсы: короче, быстрее (C-реализация), самодокументируется,
- работает с несколькими полями: itemgetter(0, 2).
- Правило: если лямбда — это ровно r[i] или r['k'],
- замените её на itemgetter.

itemgetter vs lambda

```
from operator import itemgetter

rows = [('K-12', 85.4), ('K-13', 12.7)]
recs = [{'id': 'K-12', 'q': 85.4}]

>>> sorted(rows, key=itemgetter(1))
[('K-13', 12.7), ('K-12', 85.4)]
>>> sorted(recs, key=itemgetter('q'))
[{'id': 'K-12', 'q': 85.4}]

# несколько полей сразу:
>>> sorted(rows, key=itemgetter(1, 0))
```

functools.partial: фиксация аргументов

- partial(f, ...) создаёт новую функцию с «зашитыми»
- частью аргументов.
- oil_default = partial(oil_rate, wct=0.30) —
- теперь вызов короче: oil_default(100.0).
- Плюсы перед lambda: работает с любыми аргументами,
- сохраняет имя и документацию исходной функции.
- Типовые применения: пресеты настроек, обработчики
- с фиксированным контекстом, адаптация сигнатуры
- под ожидания библиотеки.

консоль Python

```
from functools import partial

def oil_rate(q_liq, wct):
    return round(q_liq * (1 - wct), 2)

>>> by30 = partial(oil_rate, wct=0.30)
>>> by30(100.0)
70.0
>>> by30(200.0)
140.0

# то же через lambda:
>>> by30b = lambda q: oil_rate(q, 0.30)
>>> by30b(100.0)
70.0
```

Генераторные выражения vs map/filter

- Одно и то же можно записать тремя способами.
- map/filter: «функционально», коротко для простой операции.
- Генератор: (f(x) for x in xs if p(x)) — читается как фраза,
- поддерживает условие и преобразование в одном месте.
- Оба ленивы: не создают промежуточных списков —
- важно для больших данных.
- Практика: в аналитическом коде чаще используют
- генераторы и включения — их проще читать.
- Список нужен явно — оберните в list().

```
три способа одного
qs = [120.0, 0.0, 80.0]

# 1. map + filter
list(map(lambda q: q * 0.65,
         filter(lambda q: q > 0, qs)))

# 2. генератор (ленивый)
(q * 0.65 for q in qs if q > 0)

# 3. включение списка
[q * 0.65 for q in qs if q > 0]

# все три: [78.0, 52.0]
```

Композиция и конвейер

- Композиция: compose(f, g)(x) = f(g(x)) — «сначала g, потом f».
- Функция возвращает функцию — это тоже функция высшего порядка.
- Конвейер обработки данных: очистка → расчёт → фильтр →
- формат; каждый шаг — маленькая чистая функция.
- Плюсы: шаги переиспользуются и тестируются отдельно,
- порядок виден в одной строке.
- Так же устроены pandas-пайплайны (.pipe) и обработчики
- в ETL.

композиция

```
def compose(f, g):
    return lambda x: f(g(x))

def clean(q):  return max(q, 0.0)
def oil(q):    return round(q * 0.65, 2)

>>> pipeline = compose(oil, clean)
>>> pipeline(-5.0), pipeline(120.0)
(0.0, 78.0)

# конвейер по списку:
>>> [pipeline(q) for q in [-5.0, 120.0]]
[0.0, 78.0]
```

Мини-проект: функции для дебита

- Собираем всё вместе: чистая функция расчёта, валидация,
- применение к списку записей, сортировка по результату.
- oil_rate(q_liq, wct) — чистая, с docstring и аннотациями.
- validate(rec) — guard clauses, возвращает причину отказа.
- report(rows) — конвейер: валидация → расчёт → сортировка.
- Всё тестируется по частям: каждая функция маленькая
- и ничего не знает о других.
- Это и есть «прототипирование ИТ-решения» из названия курса.

готовый модуль

```
def oil_rate(q_liq, wct=0.0):
    if q_liq <= 0: return 0.0
    return round(q_liq * (1 - wct), 2)

def build(rows):
    out = []
    for wid, q, w in rows:
        out.append((wid, oil_rate(q, w)))
    return sorted(out, key=lambda r: r[1],
                  reverse=True)

>>> build([('K-12', 120.0, 0.35), ('K-13', 80.0,
0.5)])
[('K-12', 78.0), ('K-13', 40.0)]
```

Аргументы: default и порядок

- Изменяемый default (list/dict/set) накапливает данные
- между вызовами — используйте None.
- Перепутанный порядок позиционных аргументов не даёт ошибки:
- два float — и результат «тихо неверный».
- Лечение: именованные аргументы в вызове,
- особенно если рядом два числа одного смысла.
- Default не спасает от None: явная передача None
- перекрывает default — проверяйте на None внутри.
- Ловушка: f(x, y=[]) + f(x, y=None) в одном модуле —
- разное поведение при одинаковом вызове.

три ошибки рядом

```
# 1. mutable default
def push(x, acc=[]):     # накопит

# 2. порядок не тот — молча
oil_rate(0.35, 120.0)    # -41.65

# 3. None перекрывает default
def f(q, w=0.3):
    return q * (1 - w)
>>> f(100.0, None)
TypeError: unsupported operand
```

Области видимости: затенение

- Присваивание внутри функции делает имя ЛОКАЛЬНЫМ
- на всю функцию — даже если строка с присваиванием ниже.
- Отсюда UnboundLocalError при чтении до присваивания.
- Затенение встроенных: sum = 0 ломает sum() до конца модуля.
- Затенение глобальных: локальная переменная с тем же именем
- «прячет» глобальную — и правки не видны снаружи.
- Лечение: уникальные имена, отсутствие global,
- явная передача параметров.

```
консоль Python
x = 10

def f():
    print(x)     # читаем до присваивания
    x = 5        # но x локальный!

>>> f()
UnboundLocalError: cannot access local
variable 'x' where it is not associated

# и затенение built-in:
>>> sum = 0
>>> sum([1, 2])
TypeError: 'int' object is not callable
```

lambda и позднее связывание

- lambda в цикле захватывает ПЕРЕМЕННУЮ, а не её значение —
- все функции видят последнее значение.
- Лечение: lambda i=i: i (значение в default).
- Вторая форма — фабрика: (lambda n: lambda: n)(i).
- Третья — не создавать lambda в цикле вовсе:
- собрать def-функции или список кортежей.
- Та же ловушка у замыканий и у обработчиков событий.

три решения

```
# проблема:
>>> fs = [lambda: i for i in range(3)]
>>> [f() for f in fs]
[2, 2, 2]

# 1. значение в default:
>>> fs = [lambda i=i: i for i in range(3)]
[0, 1, 2]

# 2. фабрика:
>>> mk = lambda n: (lambda: n)
>>> [f() for f in map(mk, range(3))]
[0, 1, 2]
```

Типичные ошибки и ловушки

Нет return — функция вернёт None; print ≠ return.

return a, b — это кортеж, а не «два значения».

acc=[] в def — накапливает между вызовами; ставьте None.

Порядок аргументов — два float перепутать легко — зовите по имени.

sum = 0 — затенили встроенную функцию; своих имён — не built-in.

x += 1 в замыкании — без nonlocal — UnboundLocalError.

lambda в цикле — позднее связывание: lambda i=i: i.

key=lambda r: r[1] — для «взять поле» короче itemgetter(1).

Итоги лекции

1. def создаёт объект-функцию; тело выполняется только при вызове.
2. return отдаёт результат; без return функция возвращает None, print — не return.
3. Позиционные и именованные аргументы; default вычисляется один раз при def.
4. *args — кортеж позиционных, **kwargs — словарь именованных; *seq/**d при вызове.
5. Порядок: позиционные → default → *args → keyword-only → **kwargs; / и * управляют видом.
6. LEGB: Local → Enclosing → Global → Built-in; global/nonlocal — только для записи.
7. lambda — одно выражение без имени: для key= и колбэков; в цикле — lambda i=i: i.
8. Функции — значения: map/filter/reduce, itemgetter, partial, композиция.

Вопрос 1 из 6

Чем return отличается от print? Что вернёт функция без return?

Вопрос 2 из 6

Чем *args отличается от **kwargs и в каком порядке они стоят в сигнатуре?

Вопрос 3 из 6

Почему def add(x, acc=[]) — плохая идея и как это исправить?

Вопрос 4 из 6

Объясните правило LEGB: где Python ищет имя и почему присваивание делает имя локальным?

Вопрос 5 из 6

Что вернёт [f() for f in [lambda: i for i in range(3)]] и как получить [0, 1, 2]?

Вопрос 6 из 6

Чем lambda отличается от def и когда lambda уместна?